

Lab 3 - Vic Grigoryev

1. In Minix (or any other Unix), if user 2 links to a file owned by user 1, then user 1 removes the file, what happens when user 2 tries to read the file? (Tanenbaum and Woodhull, Ch. 1, Ex. 15)

The file remains, and user 2 is able to open it with the permissions they currently have. Linking a file simply creates a new file with the same file descriptor, just in a different directory than the original. Functionally, this is like duplicating a file's entrypoint, so when the file is modified both entry points reflect the same changes. Only when all file descriptors are no longer present will Unix remove that file from the disk and actually delete it.

2. Under what circumstances is multiprogramming likely to increase CPU utilization? Why?

On computers that don't use I/O often or when I/O is not a large fraction of the CPU wait time, multiprogramming may cause improvements but not significantly. However, on computers that process lots of data and read off of the I/O devices a significant amount of times for a while, multiprogramming keeps the CPU busy while the I/O is taking place so as to not waste the I/O time when instructions could still be executed.

3. Suppose a computer can execute 1 billion instructions/sec and that a system call takes 1000 instructions, including the trap and all the context switching. How many system calls can the computer execute per second and still have half the CPU capacity for running application code? (T&W 1-21)

500mil/sec saved for system calls. A system call takes 1000 instructions. $500\text{mil}/1000 = 500,000$ system calls per second.

4. What is a race condition? What are the symptoms of a race condition?(T&W 2-9)

A race condition is when two or more processes are communicating via shared data and the result depends on which order or when they run. Because these happen under smaller circumstances, ex: an interrupt during one of the programs, they are harder to debug and identify. However, when this happens it usually results in the program hanging or asleep.

5. Does the busy waiting solution using the turn variable (Fig. 2-10 in T&W) work when the two processes are running on a shared-memory multiprocessor, that is, two CPUs, sharing a common memory? (T&W, 2-13)

Busy waiting in general wastes CPU time. Sharing a common memory allows them to share the turn variable and properly adjust it, but it still results in either one of the CPUs waiting for the other to finish its noncritical and critical region to set turn correctly if one of the processes is faster than the other. It eliminates race conditions but it does not use the CPU efficiently, and violates the “one process in the critical area at a time” rule, since the faster process is waiting on a process not in the critical area. Putting the processes into two different CPUs makes no difference.

6. Describe how an operating system that can disable interrupts could implement semaphores. That is, what steps would have to happen in which order to implement the semaphore operations safely. (T&W, 2-10)

A semaphore is a variable that has two operations - down and up. Downing a semaphore causes all future processes that try to down it to go to sleep, and upping a semaphore wakes up any processes trying to access it. Since the semaphore instructions must not be interrupted, an operating system that can disable interrupts can do so during the semaphore instruction, then enable them when it is finished.

For example, a process A downs a semaphore. This would disable interrupts on the system level and run instructions to decrement the semaphore counter, after which a process B tries to access the semaphore. During this set of non interrupted instructions, the semaphore cannot decrement and puts B to sleep. After some time, process A ups the semaphore, causing a non interruptible set of instructions to increment the semaphore counter, which also wakes up process B and lets it continue.

7. Round robin schedulers normally maintain a list of all runnable processes, with each process occurring exactly once in the list. What would happen if a process occurred twice in the list? Can you think of any reason for allowing this? (T&W, 2-25) (And what is the reason. “Yes” or “no” would not be considered a sufficient answer.)

If a process occurred twice in the round robin list, this would just mean it runs one more time than the other ones on this list that just occur once. This can be done to simulate tasks with a certain priority level since they run longer on this list (and if they’re the same length as the other processes, will finish faster). There shouldn’t be any other issues that arise from this arrangement.

8. Five batch jobs, A through E, arrive at a computer center, in alphabetical order, at almost the same time. They have estimated running times of 10, 3, 4, 7, and 6 seconds respectively. Their (externally determined) priorities are 3, 5, 2, 1, and 4, respectively, with 5 being the highest priority. For each of the following scheduling algorithms, determine the time at which each job completes and the mean process turnaround time. Assume a 1 second quantum and ignore process switching overhead. (Modified from T&W, 2-28)

For (a), assume that the system is multiprogrammed, and that each job gets its fair share of the CPU. For (b)–(d) assume that only one job at a time runs, and each job runs until it is finished. All jobs are completely CPU bound.

A	B	C	D	E
10	3	4	7	6
3	5	2	1	4

(a) Round robin.

A, 10 -> B, 3 -> C, 4 -> D, 7 -> E, 6

A, 9 -> B, 2 -> C, 3 -> D, 6 -> E, 5

A, 8 -> B, 1 -> C, 2 -> D, 5 -> E, 4

A, 7 -> C, 1 -> D, 4 -> E, 3

A, 6 -> D, 3 -> E, 2

A, 5 -> D, 2 -> E, 1

A, 4 -> D, 1

A, 3

A, 2

A, 1

A finishes in 30 seconds

B finishes in 12 seconds

C finishes in 17 seconds

D finishes in 27 seconds

E finishes in 25 seconds

Mean: 22.2 seconds

(b) Priority scheduling.

B, 3 -> B, 2 -> B, 1

E, 6 -> E, 5 -> E, 4 -> E, 3 -> E, 2 -> E, 1

A, 10 -> A, 9 -> A, 8 -> A, 7 -> A, 6 -> A, 5 -> A, 4 -> A, 3 -> A, 2 -> A, 1

C, 4 -> C, 3 -> C, 2 -> C, 1

D, 7 -> D, 6 -> D, 5 -> D, 4 -> D, 3 -> D, 2 -> D, 1

A finishes in 19 seconds

B finishes in 3 seconds

C finishes in 23 seconds

D finishes in 30 seconds

E finishes in 9 seconds

Mean: 16.8 seconds

(c) First-come, first served (given that they arrive in alphabetical order).

A, 10 -> A, 9 -> A, 8 -> A, 7 -> A, 6 -> A, 5 -> A, 4 -> A, 3 -> A, 2 -> A, 1

B, 3 -> B, 2 -> B, 1

C, 4 -> C, 3 -> C, 2 -> C, 1

D, 7 -> D, 6 -> D, 5 -> D, 4 -> D, 3 -> D, 2 -> D, 1

E, 6 -> E, 5 -> E, 4 -> E, 3 -> E, 2 -> E, 1

A finishes in 10 seconds

B finishes in 13 seconds

C finishes in 17 seconds

D finishes in 24 seconds

E finishes in 30 seconds

Mean: 18.8 seconds

(d) Shortest job first.

B, 3 -> B, 2 -> B, 1

C, 4 -> C, 3 -> C, 2 -> C, 1

E, 6 -> E, 5 -> E, 4 -> E, 3 -> E, 2 -> E, 1

D, 7 -> D, 6 -> D, 5 -> D, 4 -> D, 3 -> D, 2 -> D, 1

A, 10 -> A, 9 -> A, 8 -> A, 7 -> A, 6 -> A, 5 -> A, 4 -> A, 3 -> A, 2 -> A, 1

A finishes in 30 seconds

B finishes in 3 seconds

C finishes in 7 seconds

D finishes in 20 seconds

E finishes in 13 seconds

Mean: 14.6 seconds

9. Re-do problem 8a with the modification that job D is IO bound. After each 500ms it is allowed to run, it blocks for an IO operation that takes 1s to complete. The IO processing itself doesn't take any noticeable time. Assume that jobs moving from the blocked state to the ready state are placed at the end of the run queue. If a blocked job becomes runnable at the same time a running process's quantum is up, the formerly blocked job is placed back on the queue ahead of the other one.

A B C D E
 10 3 4 7 6
 3 5 2 1 4

(a) Round robin.

A, 10 -> B, 3 -> C, 4 -> D, 7 (block, 0.5s) -> E, 6

A, 9 -> B, 2 -> C, 3 -> D, 6 (block, 0.5s) -> E, 5

A, 8 -> B, 1 -> C, 2 -> D, 5 (block, 0.5s) -> E, 4

A, 7 -> C, 1 -> D, 4 (block, 0.5s) -> E, 3

A, 6 -> D, 3 (block, 0.5s) -> E, 2

A, 5 -> D, 2 (block, 0.5s) -> E, 1

A, 4 -> D, 1 (block, 0.5s)

A, 3

A, 2

A, 1

A finishes in 26.5 seconds

B finishes in 11 seconds

C finishes in 15.5 seconds

D finishes in 23.5 seconds

E finishes in 22 seconds

Mean: 19.7 seconds

(b) Priority scheduling.

B, 3 -> B, 2 -> B, 1

E, 6 -> E, 5 -> E, 4 -> E, 3 -> E, 2 -> E, 1

A, 10 -> A, 9 -> A, 8 -> A, 7 -> A, 6 -> A, 5 -> A, 4 -> A, 3 -> A, 2 -> A, 1

C, 4 -> C, 3 -> C, 2 -> C, 1

D, 7 (blocks, 1.5s) -> D, 6 (blocks, 1.5s) -> D, 5 (blocks, 1.5s) -> D, 4 (blocks, 1.5s) -> D, 3 (blocks, 1.5s) -> D, 2 (blocks, 1.5s) -> D, 1 (blocks, 1.5s)

A finishes in 19 seconds

B finishes in 3 seconds

C finishes in 23 seconds

D finishes in 33.5 seconds

E finishes in 9 seconds

Mean: 17.5 seconds

(c) First-come, first served (given that they arrive in alphabetical order).

A, 10 -> A, 9 -> A, 8 -> A, 7 -> A, 6 -> A, 5 -> A, 4 -> A, 3 -> A, 2 -> A, 1

B, 3 -> B, 2 -> B, 1

C, 4 -> C, 3 -> C, 2 -> C, 1

D, 7 (blocks, 0.5s) -> E, 6 -> D, 6 (blocks, 0.5s) -> E, 5 -> D, 5 (blocks, 0.5s) -> E, 4 -> D, 4 (blocks, 0.5s) -> E, 3 -> D, 3 (blocks, 0.5s) -> E, 2 -> D, 2 (blocks, 0.5s) -> E, 1 -> D, 1 (blocks, 0.5s)

A finishes in 10 seconds

B finishes in 13 seconds

C finishes in 17 seconds

D finishes in 26.5 seconds

E finishes in 26 seconds

Mean: 18.5 seconds

(d) Shortest job first.

B, 3 -> B, 2 -> B, 1

C, 4 -> C, 3 -> C, 2 -> C, 1

E, 6 -> E, 5 -> E, 4 -> E, 3 -> E, 2 -> E, 1

D, 7 (blocks, 0.5s) -> A, 10 -> D, 6 (blocks, 0.5s) -> A, 9 -> D, 5 (blocks, 0.5s) -> A, 8 -> D, 4 (blocks, 0.5s) -> A, 7 -> D, 3 (blocks, 0.5s) -> A, 6 -> D, 2 (blocks, 0.5s) -> A, 5 -> D, 1 (blocks, 0.5s) -> A, 4 -> A, 3 -> A, 2 -> A, 1

A finishes in 26.5 seconds

B finishes in 3 seconds

C finishes in 7 seconds

D finishes in 22.5 seconds

E finishes in 13 seconds

Mean: 14.4 seconds

10. A CPU-bound process running on CTSS needs 30 quanta to complete. How many times must it be swapped in, including the first time (before it has run at all)? Assume that there are always other runnable jobs and that the number of priority classes is unlimited. (T&W, 2-29)

1. 1 quantum, 29 remaining
2. 2 quanta, 27 remaining
3. 4 quanta, 23 remaining
4. 8 quanta, 15 remaining
5. 16 quanta, finished

5 swaps to finish the process